

Использование корутин для исполнения методов БЛ

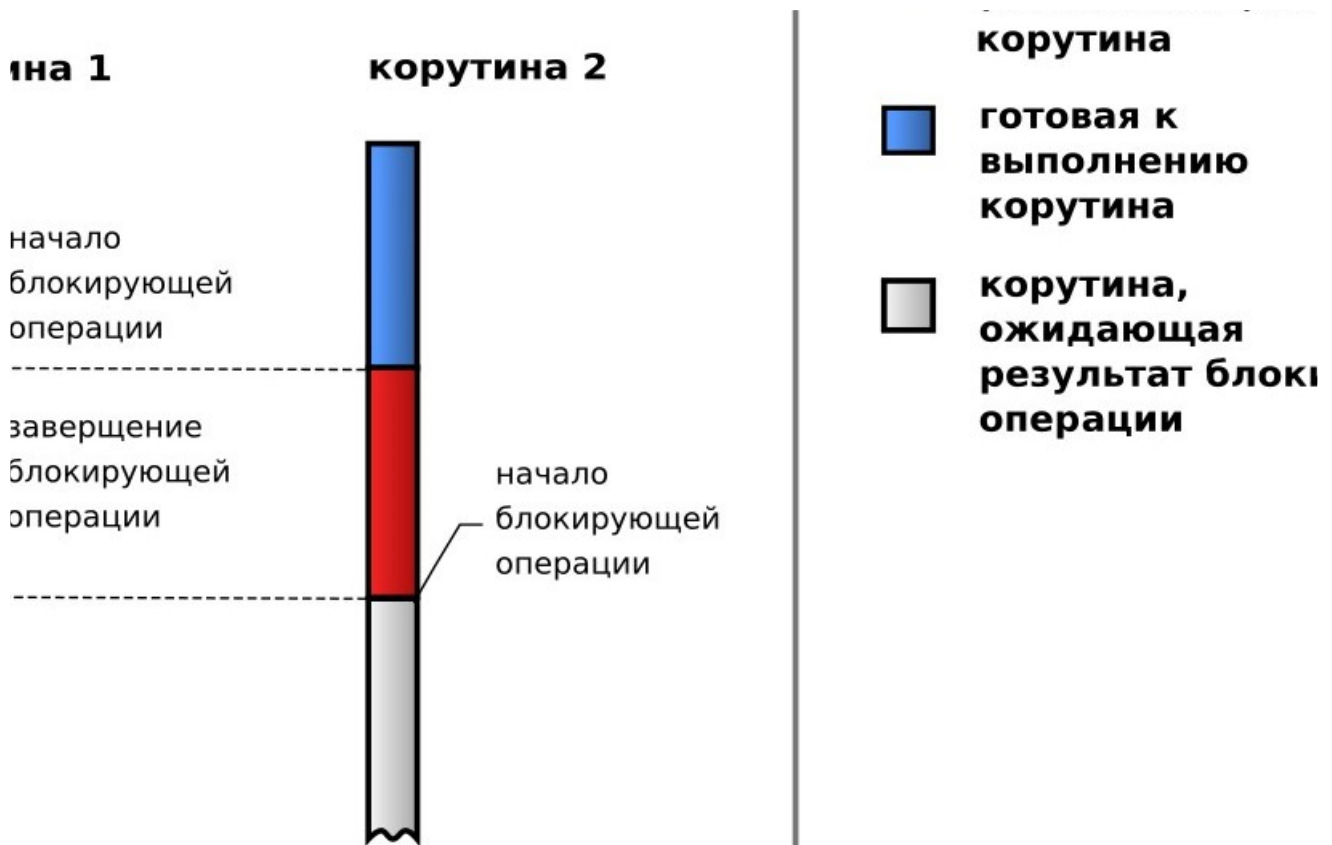
В Платформе появилась поддержка [stackful-корутин](#) и возможность исполнять в них [методы бизнес-логики](#). Stackful-корутины можно рассматривать как легковесные пользовательские потоки, имеющие собственный стек и использующие [кооперативную многозадачность](#).

Сервер приложений предполагает исполнение метода БЛ в каком-либо из рабочих потоков пула исполнителей (потоке встроенного пула процесса-координатора или в потоке рабочего процесса), при этом текущая архитектура сервера и оформление методов бизнес-логики используют синхронную модель исполнения.

Таким образом, любой входящий запрос на всё время своей обработки эксклюзивно владеет контекстом исполнения (потоком).

Число потоков - исполнителей, имеющихся в распоряжении рабочего процесса (или встроенного пула процесса-координатора), ограничивает количество задач, которые могут одновременно выполняться в процессе, что делает поток крайне ценным ресурсом. В рамках своей работы метод бизнес - логики может производить блокирующие операции (например, синхронный вызов удаленного метода). На время ожидания завершения блокирующей операции поток, исполняющий метод бизнес-логики, по сути простаивает.

Чтобы избежать простоя потока, можно организовать выполнение методов БЛ в корутинах. В таком случае схема обработки запросов модифицируется следующим образом: в потоке создается от одной до N корутин, каждая из которых может обрабатывать запрос (исполнять метод БЛ). Если текущая активная корутина в процессе выполнения метода бизнес-логики столкнулась с блокирующей операцией, можно приостановить исполнение корутины на время ожидания завершения этой операции и передать управление другой корутине, которая в это время может исполнить другой метод бизнес-логики. Схематично этот процесс изображен на рисунке.



Процесс создания новых корутин в потоке, выбора следующей активной корутины и передачи ей управления, а также завершения (лишних) простаивающих корутин должен координироваться некоторой сущностью. В её роли выступает планировщик корутин.

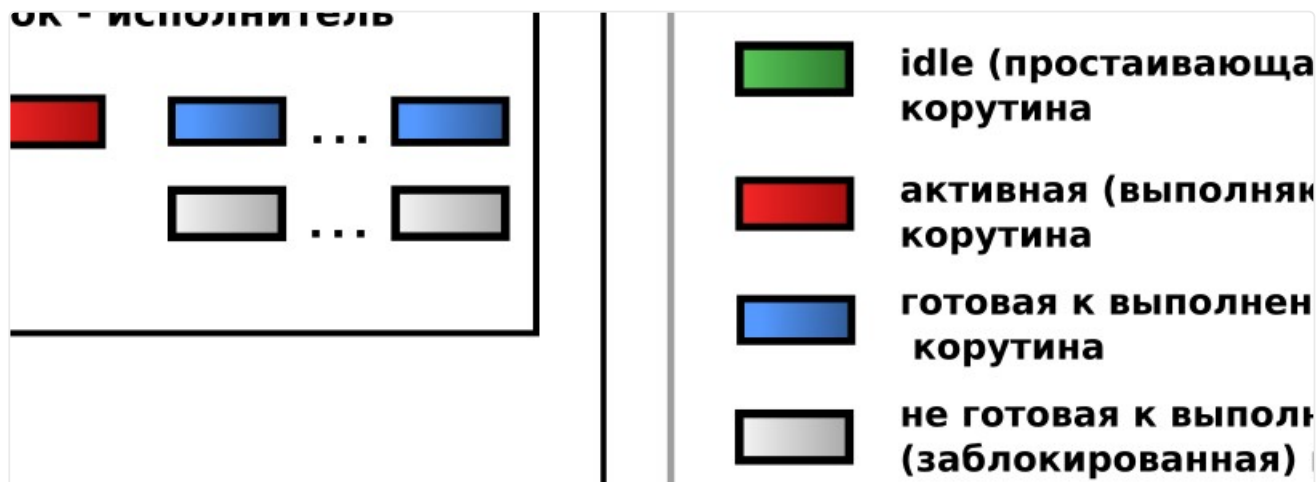
В общих чертах планировщик создает новую корутину и передает ей управление. Получив в какой-то момент времени управление обратно от корутины (напомним, корутины используют кооперативную многозадачность и должны явно уступать управление), планировщик принимает решение о судьбе корутины, от которой получил управление (поместить в список готовых к исполнению корутин, поместить в лист ожидания, завершить), и назначает следующую активную корутину (может быть создана новая корутина, выбрана готовая для исполнения корутина или планировщик

может дожидаться готовности какой-либо корутины). Завершена может быть только простаивающая корутина (т. е. корутина, которая не обрабатывает в данный момент какой-либо запрос).

Есть два типа планировщиков корутин: `template` и `dedicated`. Их описание представлено ниже.

Тип планировщика корутин `template`

Планировщик корутин `template` создает до N корутин в потоке-исполнителе. Т. е. одновременно в потоке может обрабатываться до N запросов. Все готовые к исполнению корутины считаются равноправными, переключение между ними производится по принципу [FIFO](#).



Планировщик поддерживает в наличии не более одной простаивающей корутины. Лишние простаивающие корутины завершаются; в случае отсутствия простаивающих корутин создается новая корутина (если не достигнут предел N на количество корутин в потоке). Планировщик `template` по умолчанию разрешает корутинам уступать управление. Т. е. в любой точке переключения (точке `yield`) будет передано управление планировщику, если это явно не запрещено в коде, исполняемом корутиной.



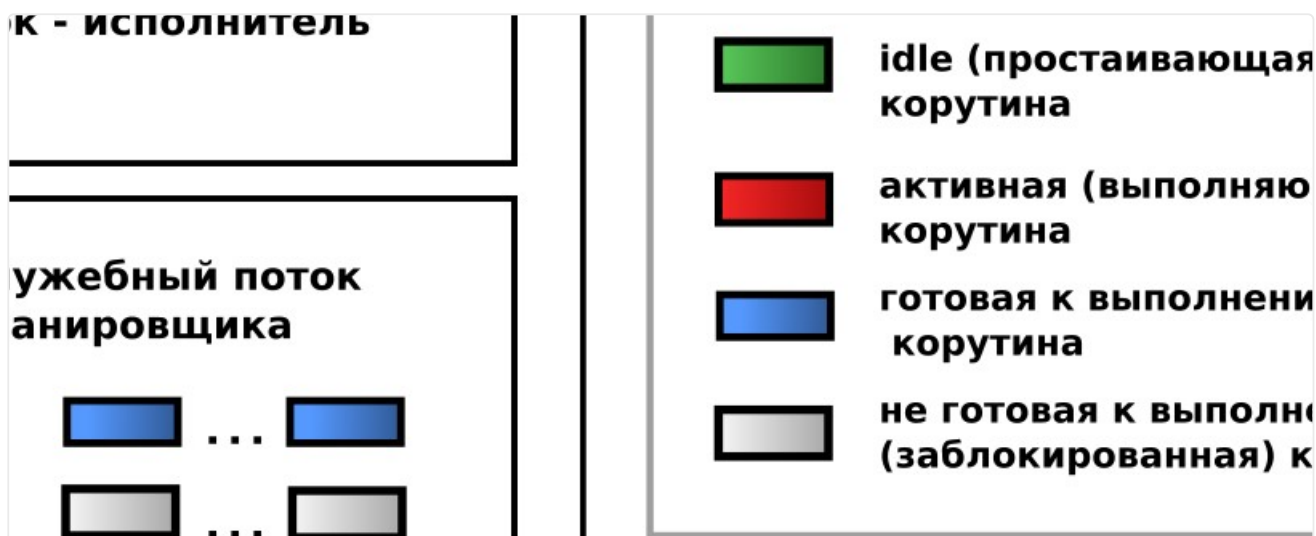
В Платформе версии 21.2100 есть только один контекст, в котором запрещено переключать корутины: если в стеке вызовов присутствуют фреймы, принадлежащие интерпретатору `javascript` (`libv8`). Запрет на переключение связан с тем, что интерпретатор активно использует `tss`.

Схема работы планировщика `template` предполагает, что все обрабатываемые корутинами запросы готовы к кооперативной многозадачности (не содержат длительных по времени исполнения участков кода без точек переключения).

Тип планировщика корутин `dedicated`

Планировщик корутин `dedicated` всегда имеет ровно одну корутину в потоке-исполнителе (простаивающую или обрабатывающую запрос) и до N корутин в собственном служебном потоке.

Схема работы планировщика `dedicated` предполагает, что только некоторые запросы готовы к кооперативной многозадачности.



Новые запросы всегда обрабатывает корутина, расположенная в потоке-исполнителе. Если в процессе обработки запроса корутина вернула управление планировщику, то планировщик делает вывод, что данный запрос готов к кооперативной многозадачности. В этом случае планировщик передаёт корутину в свой служебный поток, в котором корутина продолжает выполнение запроса по правилам кооперативности, а вместо неё в потоке-исполнителе создаёт новую корутину, готовую к обработке следующего запроса. Всего в служебном потоке может находиться до N корутин. При этом все готовые к исполнению корутины считаются равноправными, переключение между ними производится по принципу FIFO. Простаивающие корутины (закончившие выполнение запроса) в служебном потоке завершаются планировщиком.

Таким образом, одновременно под управлением планировщика может выполняться не более одной задачи, не готовой к кооперативной многозадачности (эксклюзивно в потоке-исполнителе), и не более N задач, готовых к кооперативной многозадачности (в служебном потоке планировщика). Новая задача поступает на исполнение, только если в настоящий момент не исполняется задача, не готовая к кооперативной многозадачности.

Чтобы в служебный поток передавались только корутины, выполняющие запрос, готовый к кооперативной многозадачности, планировщик `dedicated` по умолчанию запрещает корутинам уступать управление. Т. е. в точке переключения (точке `yield`) управление будет передано планировщику, только если это явно разрешено в коде, исполняемом корутиной.



Есть только один контекст, в котором явно разрешено переключать корутины: исполнение прокси-метода.

Настройка корутин для использования в сервисе

Механизм корутин по умолчанию не используется. Чтобы включить использование корутин для какого-либо сервиса, необходимо задать соответствующие параметры в облаке.

- ### Ядро.Сервер приложений.ТипПланировщикаКорутин

Задаёт реализацию планировщика корутин. Если значение не установлено, корутины не используются. В Платформе имеются следующие реализации планировщика корутин: [dedicated](#), [template](#).

- ### Ядро.Сервер приложений.ОграничениеНаЧислоКорутин

Ограничение на число корутин, работающих под управлением экземпляра планировщика корутин (максимальное количество корутин, выполняющееся в одном потоке). Если установлено значение, меньшее либо равное нулю, корутины не используются.

В описании типов планировщика корутин, представленном выше, это число фигурирует как N.

- ### Ядро.Сервер приложений.МаксимальноеЧислоВнутреннихКорутин

Максимальное число внутренних корутин, работающих под управлением экземпляра планировщика корутин при выполнении в потоке корутин (максимальное количество корутин, выполняющееся в одном потоке). Если установлено значение, меньшее либо равное нулю, корутины не используются. Данный параметр используется если создан отдельный поток со своим планировщиком для выполнения поставленных задач.

Контролирует исполнение асинхронных вызовов методов (`FutureInvoke`).

Кроме этого существуют опциональные параметры, которые можно использовать для настройки стеков корутин.

- ### Ядро.Сервер приложений.РазмерСтекаКорутины

Размер стека корутины в килобайтах. Если установлено значение 0, используется размер стека по умолчанию.

Стек корутины является аналогом стека потока. Каждая корутина обладает своим стеком фиксированного размера.

- **Ядро.Сервер приложений.РазмерПулаСтековКорутин**

Максимальный размер пула (кэша) стеков корутин. Процесс держит в резерве не более указанного количества стеков (если установлено значение 0, то стеки не держатся в резерве (выделяются/освобождаются при создании/завершении корутины); если указано значение -1, то размер пула не ограничен). Значением по умолчанию является -1.

В дополнение на работу корутин, в рамках исполнения которых производятся запросы в БД, влияют параметры пула соединений с БД PostgreSQL.

- **Ядро.База данных.РазмерКешаСоединений**

Максимальное количество закешированных логических соединений с БД.

- **Ядро.База данных.РазмерПереполненияКешаСоединений**

Максимальное количество логических соединений с БД, которые могут одновременно быть выданы при исчерпании кэша соединений (сверх размера кэша).

При указании отрицательного значения количество соединений ограничиваться не будет.

Выбор значений параметров

В данном разделе приведены общие рекомендации по настройке механизма корутин с типом планировщика [template](#).

При переходе на кооперативную многозадачность необходимо сократить количество рабочих (логических потоков), которое при использовании механизма вытесняющей многозадачности, как правило, значительно превышает количество исполнителей (аппаратных потоков/ядер). Рекомендуется выставлять параметры [Ядро.Сервер приложений.ЧислоРабочихПроцессов](#), [Ядро.Сервер приложений.ЧислоСлужебныхРабочихПроцессов](#), [Ядро.Сервер приложений.ЧислоПотоковВРабочихПроцессах](#), [Ядро.Сервер приложений.ЧислоПотоковВСлужебныхРабочихПроцессах](#) таким образом, чтобы суммарное количество рабочих потоков на узле БЛ соответствовало количеству аппаратных потоков/ядер; а параметр [Ядро.Сервер приложений.ЧислоДополнительныхПроцессов](#) сбросить на 0.

Параметры количества корутин, [Ядро.Сервер приложений.ОграничениеНаЧислоКорутин](#) и [Ядро.Сервер приложений.МаксимальноеЧислоВнутреннихКорутин](#), следует выставлять, исходя из типичной пиковой нагрузки сервиса: корутин должно быть достаточно для полной аппаратной загрузки БЛ. Однако количество корутин не должно быть слишком большим, чтобы конкуренция за ограниченный внешний ресурс (к примеру, за БД) не приводила к повышенной нагрузке на другие ресурсы, в особенности на оперативную память (стеки корутин, локальные данные методов БЛ). Рекомендуемое начальное приближение: 200 корутин.

Для сервисов, использующих БД PostgreSQL, требуется увеличить значение параметра [Ядро.База данных.РазмерКешаСоединений](#), учитывая кумулятивную нагрузку на БД со всех рабочих процессов всех узлов БЛ. Рекомендуемое начальное приближение: 50 соединений (для конфигурации с 2 узлами БЛ по 4 рабочих в каждом). Параметру [Ядро.База данных.РазмерПереполненияКешаСоединений](#) следует выставить неотрицательное значение: в общем случае рекомендуется нулевое значение, в качестве альтернативы можно задать долю от значения параметра [РазмерКешаСоединений](#) для сглаживания кратковременного увеличения количества запросов в БД.

Точки переключения корутин

Следующие операции являются точками потенциального переключения корутин:

- Ожидание готовности [Waitable](#)-объектов посредством семейства методов `Wait*`, вспомогательного класса [Waiter](#) и внешних функций [WaitForAll](#), [WaitForAny](#), [WaitForAllFor](#), [WaitForAnyFor](#).
- Получение результата из объекта [Future](#).
- Блокирующие операции примитивов синхронизации: [Mutex](#), [RecursiveMutex](#), [RecursiveTimedMutex](#), [Semaphore](#), [TimedMutex](#).
- Явная уступка контекста исполнения с помощью функции [YieldFiber](#).
- Операции с PostgreSQL, а также ожидание и выборка результатов операций через асинхронные механизмы [IAsyncConnect](#), [IAsyncQueryResult](#) ([C++/Python](#)), [INotifyListener](#) ([C++/Python](#)).
- Операции с Redis, а также ожидание и выборка результатов операций через асинхронные механизмы [AsyncBatchOperation](#), [AsyncOperation](#).
- Внешние вызовы.

Ограничения, накладываемые корутинами

При внедрении корутин в существующую кодовую базу, а также при реализации новой функциональности с их использованием, следует учитывать главную особенность их исполнения: корутины могут уступать поток исполнения

другим корутинам до завершения выполняемого метода БЛ, а при использовании `dedicated` планировщика ещё и мигрировать в другой поток.

Это накладывает ряд ограничений на использование функционала, прямо или косвенно связанного с текущим потоком, в коде, исполняемом в контексте корутины. Сводящихся к запрету прямого или опосредованного применения операций, потенциально переключающих корутины (см. [точки переключения корутин](#)) в следующих сценариях:

1. Из-под блокировок некорутинных объектов синхронизации (`std::mutex`, `boost::mutex` и им подобные).

При переключении корутины с захваченным мьютексом появляется вероятность самоблокировки планировщика в случае постановки на исполнение корутины, в процессе исполнения которой требуется заблокировать тот же мьютекс.

А при использовании рекурсивных мьютексов в таком сценарии возникает вероятность повторного входа в область кода, защищённую блокировкой мьютекса.

Общая рекомендация в данном случае - использовать корутинные примитивы синхронизации [Mutex](#), [RecursiveMutex](#), [RecursiveTimedMutex](#), [Semaphore](#), [TimedMutex](#).

2. Из конструкторов глобальных и статических переменных.

Это ограничение связано с тем, что вызов конструкторов таких переменных производится с заблокированным мьютексом глобальной инициализации стандартной библиотеки, что приводит к потенциальной самоблокировке планировщика, как и в сценарии (1).

3. При использовании локального хранилища потока (`thread_local` переменные) для хранения объектов, не предназначенных для совместного использования несколькими методами БЛ.

После переключения корутин состояние таких объектов может оказаться противоречивым.

Общая рекомендация в данном случае - использовать локальное хранилище корутины при помощи переменных, объявленных с применением макроса [SBIS_CONTEXT_LOCAL](#).

4. Из `catch`-блоков при перехвате исключений.

Это ограничение напрямую связано со сценарием (3), т.к. стандартная библиотека использует локальное хранилище потока в процессе раскрутки стека. В данном варианте после переключения в `catch`-блоке новая корутина "наследует" состояние раскрутки от предыдущей, потенциально спутывая его с собственным состоянием раскрутки, если в ней также выбрасывается исключение. Общая рекомендация в данном случае - выносить вызов функциональности, переключающей корутины, за `catch`-блоки.

5. Из деструкторов объектов.

Это ограничение аналогично сценарию (4), т.к. деструктор объекта может быть вызван при раскрутке стека по выбросу исключения, что влечёт риск смешивания состояния раскрутки стека стандартной библиотеки уступающей корутины с состоянием раскрутки следующей корутины.

Общая рекомендация в данном случае - выносить вызов функциональности, переключающей корутины, за деструкторы.